

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

SISTEMA WEB DE MONITOREO Y GESTIÓN DE LA INFRAESTRUCTURA DE REDES - ESPOCH

DOCUMENTO DE METÁFORA, ARQUITECTURA DIRIGIDA POR EVENTOS Y SELECCIÓN DEL STACK TECNOLÓGICO

Arquitectura Reactiva en Tiempo Real, Ecosistema de Herramientas y Justificación Técnica

Roles de Ingeniería:

- Líder Técnico y Arquitecto Backend
- Desarrollador Frontend y Diseñador UI/UX
- Especialista en Bases de Datos y QA Lead
- Ingeniero DevOps y Redes

Asignaturas y Docentes Responsables:

- Infraestructura TI: Ing. Tatiana Landivar
- Tecnología Web: Ing. Pedro Aguilar
- Conmutación y Enrutamiento: Ing. Juan Pazmiño
- Big Data: Ing. Geovanny Silva
- Diseño y Multimedia: Ing. Ramiro Bonilla

Período Académico: Quinto Semestre (PAO 5)

Documento Oficial de Ingeniería de Software (Versión 2.0 - Arquitectura EDA)

Orellana - Ecuador

Noviembre de 2026

Índice general

1. Metáfora Arquitectural del Sistema **4**

1.1. Concepto de la Metáfora: La Torre de Control de Tráfico Aéreo y el Escuadrón Centinela 4

2. Arquitectura Dirigida por Eventos (EDA) **6**

2.1. Fundamentación Técnica y Selección del Paradigma 6

2.2. Componentes del Modelo EDA en el Sistema 6

2.2.1. 1. Productores de Eventos (Event Producers) 6

2.2.2. 2. Bus Central de Eventos en Memoria (Internal Event Bus) 7

2.2.3. 3. Consumidores de Eventos (Event Consumers / Listeners) 7

2.2.4. 4. Clientes Reactivos en Tiempo Real (WebSocket Consumers) . . . 7

3. Desglose Exhaustivo del Stack Tecnológico **9**

3.1. Capa de Presentación: Frontend Web Reactivo 9

3.2. Capa de Servicios y Lógica de Negocio: Backend Node.js 10

3.3. Capa de Monitoreo y Exploración de Red: Daemon Centinela 10

3.4. Capa de Persistencia: Base de Datos NoSQL 11

3.5. Infraestructura del Host y Despliegue en Servidor Físico 12

4. Matriz de Justificación Técnica y Alternativas Descartadas **13**

5. Articulación con las Asignaturas del Semestre (PAO 5) **16**

6. Conclusiones **17**

Índice de tablas

4.1. Matriz comparativa de arquitectura, tiempo real y desarrollo web 14

4.2. Matriz comparativa de persistencia, monitoreo de red e infraestructura . . . 15

5.1. Mapeo de tecnologías con asignaturas curriculares 16

Índice de figuras

1.1. Diagrama esquemático de la metáfora arquitectural con canal reactivo . . . 5

2.1. Flujo arquitectural de la Arquitectura Dirigida por Eventos (EDA) 8

Capítulo 1

Metáfora Arquitectural del Sistema

1.1 Concepto de la Metáfora: La Torre de Control de Tráfico Aéreo y el Escuadrón Centinela

En las metodologías ágiles de ingeniería de software (como eXtreme Programming y Scrum), la *metáfora del sistema* es una visión compartida de alto nivel que permite a todos los ingenieros de desarrollo, evaluadores de calidad y operadores comprender intuitivamente cómo opera el software, sus componentes y su flujo de datos sin extraviarse en detalles de bajo nivel.

Para el presente proyecto, la metáfora rectora seleccionada es:

"La Torre de Control de Tráfico Aéreo, el Escuadrón Centinela y el Canal de Telemetría Radárica"

Bajo esta analogía operativa:

- **La Torre de Control (Frontend React 18):** Es la cabina panorámica donde los controladores (Administradores y Operadores de red del NOC) supervisan en tiempo real la posición, latencia y estado operativo de cada equipo mediante paneles gráficos reactivos, diagramas de topología y consolas de alertas acústico-visuales. La torre no sale a la pista ni interroga directamente a las aeronaves en vuelo; simplemente proyecta de forma inmediata los datos que recibe por el canal de radiofrecuencia.
- **El Canal de Telemetría Radárica (WebSockets con Socket.io):** Representa el enlace de radiofrecuencia continuo y bidireccional entre la torre y la estación terrena de radar. Elimina de raíz la necesidad de que los controladores deban llamar por teléfono cada 5 segundos para preguntar si un avión sigue en vuelo (*antipatrón de polling HTTP*). En su lugar, cualquier variación de estado o señal de alarma se propaga instantáneamente a todas las pantallas activas en cuestión de milisegundos.
- **El Centro de Despacho y Bus de Mensajería (Backend Node.js / Express con EventBus):** Es la estación central de control que coordina las operaciones. Dispone de un bus interno de eventos que recibe las señales crudas del radar, evalúa las prioridades, despacha las alertas a los operadores y atiende transacciones seguras (como la autorización de credenciales, autenticación JWT dual y parametrización de umbrales) mediante una API REST protegida.
- **El Escuadrón Centinela de Radar (Daemon de Monitoreo en Segundo Plano):** Es un servicio autónomo gobernado por el sistema operativo que patrulla de manera ininterrum-

pida el espacio aéreo (la red de área local). Emite pulsos periódicos de radar (pings ICMP ultraligeros y barridos ARP) para verificar que los equipos registrados sigan respondiendo y detectar intrusiones o nuevas terminales no inventariadas. Cuando un equipo deja de responder a 3 pulsos sucesivos, el radar emite un evento de cambio de estado hacia el bus central.

- **La Bitácora de Vuelo y Caja Negra (Base de Datos MongoDB):** Es el archivo inalterable donde se registran la telemetría periódica, el inventario de aeronaves, el historial de cambios de topología y la bitácora de incidentes para auditoría posterior, gobernado mediante índices de expiración automática (TTL) para impedir la degradación de almacenamiento.

Esta metáfora resuelve de forma natural la separación de responsabilidades exigida en el requerimiento RNF-03: *el radar centinela continúa vigilando el espacio aéreo incluso cuando todos los operadores de la torre de control cierran sus navegadores web*.

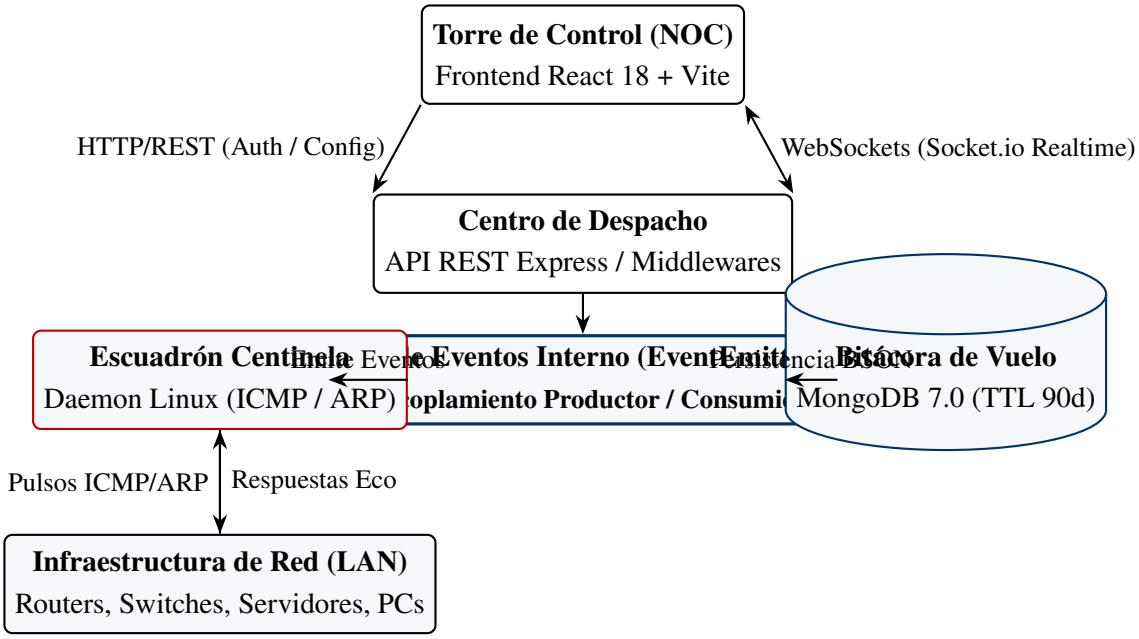


Figura 1.1: Diagrama esquemático de la metáfora arquitectural con canal reactivo

Capítulo 2

Arquitectura Dirigida por Eventos (EDA)

2.1 Fundamentación Técnica y Selección del Paradigma

Los sistemas de monitoreo y supervisión de infraestructuras críticas operan bajo una premisa fundamental: *el estado de la red cambia de manera asíncrona, espontánea y no predecible*. Una interfaz web tradicional basada en peticiones periódicas (*polling* HTTP cada 5 segundos) presenta tres deficiencias técnicas graves:

1. **Latencia de Notificación Inaceptable:** Si un switch de núcleo colapsa a los 100 ms de haber concluido un ciclo de sondeo, el operador humano no se entera hasta 4.9 segundos después, violando el umbral de respuesta inmediata exigido en salas NOC.
2. **Sobrecarga Innecesaria de E/S y Red:** Miles de peticiones HTTP vacías sobrecargan la capa de transporte TCP, consumiendo ancho de banda y ciclos de CPU en el servidor al serializar y parsear encabezados HTTP redundantes sin que haya cambios que reportar.
3. **Acoplamiento Temporal:** Obliga al backend a mantener bloqueadas conexiones o sincronizar tiempos entre módulos de escaneo y controladores web.

Para resolver estas limitaciones, se adopta formalmente una **Arquitectura Modular Dirigida por Eventos (EDA - Event-Driven Architecture)**. Este patrón separa la generación de datos de su consumo, permitiendo que la aplicación reaccione instantáneamente ante cualquier variación en la infraestructura.

2.2 Componentes del Modelo EDA en el Sistema

El modelo de eventos implementado en el sistema se articula a través de cuatro componentes bien definidos:

2.2.1 1. Productores de Eventos (Event Producers)

Son aquellos módulos que detectan cambios de estado o procesan acciones del sistema y generan señales tipadas sin conocer qué entidad las procesará posteriormente:

- **Proceso Centinela (Daemon de Red):** Emite eventos como `NETWORK_PING_RESULT` tras cada ciclo de eco ICMP, `DEVICE_DISCOVERED` cuando el barrido ARP identifica una dirección MAC desconocida, y `DEVICE_STATUS_CHANGED` cuando un nodo transiciona entre estados (`online`, `warning`, `offline`).
- **Módulo de Seguridad y Autenticación:** Emite eventos como `AUTH_LOGIN_SUCCESS`,

`AUTH_LOGIN_FAILED` y `AUTH_ACCOUNT_LOCKED` tras exceder los 5 intentos fallidos normados en la especificación.

- **Gestor de Inventario:** Emite notificaciones de creación, actualización o alteración de enlaces de topología (`DEVICE_CREATED`, `DEVICE_UPDATED`, `TOPOLOGY_LINK_CHANGED`).

2.2.2 2. Bus Central de Eventos en Memoria (Internal Event Bus)

Consiste en un despachador centralizado basado en la clase nativa `EventEmitter` de Node.js, configurado bajo el patrón *Singleton*. Sus características técnicas son:

- **Procesamiento No Bloqueante:** Las suscripciones se ejecutan en el bucle de eventos (*Event Loop*) asíncrono, garantizando que un fallo en un suscriptor jamás bloquee a los demás ni afecte al productor.
- **Catálogo Tipado de Eventos:** Se prohíbe el uso de cadenas de texto arbitrarias. Todos los eventos se definen como constantes inmutables en un módulo de constantes (`events.constants.js`).
- **Aislamiento de Errores:** Cada manejador de eventos (*listener*) encapsula su lógica en bloques controlados para que excepciones no capturadas no provoquen la terminación del proceso Node.js (`uncaughtException`).

2.2.3 3. Consumidores de Eventos (Event Consumers / Listeners)

Módulos especializados que se suscriben al bus para reaccionar de forma independiente ante eventos específicos:

- **Motor de Evaluación de Alertas (Alert Engine):** Escucha `NETWORK_PING_RESULT` y `DEVICE_STATUS_CHANGED`. Si un nodo acumula 3 fallos consecutivos o supera el umbral crítico de latencia configurado, evalúa las reglas de degradación y emite la alarma `ALERT_TRIGGERED`.
- **Servicio de Persistencia y Métricas (Metrics Store):** Escucha los eventos de telemetría y los persiste en la base de datos MongoDB aplicando agregaciones en segundo plano para no saturar el almacenamiento.
- **Pasarela WebSockets (Socket.io Realtime Gateway):** Transforma los eventos internos del backend en tramas WebSocket dirigidas a las salas activas de clientes web (`dashboard`, `alerts`, `topology`).
- **Bitácora de Auditoría (Audit Logger):** Registra los eventos de seguridad y cambios administrativos en el archivo de logs del sistema mediante Winston.

2.2.4 4. Clientes Reactivos en Tiempo Real (WebSocket Consumers)

El frontend React mantiene un canal WebSocket persistente con la librería `socket.io-client`. Al recibir un mensaje del servidor, los hooks reactivos actualizan directamente el almacén de estado y el grafo interactivo de nodos sin requerir ninguna recarga de página.

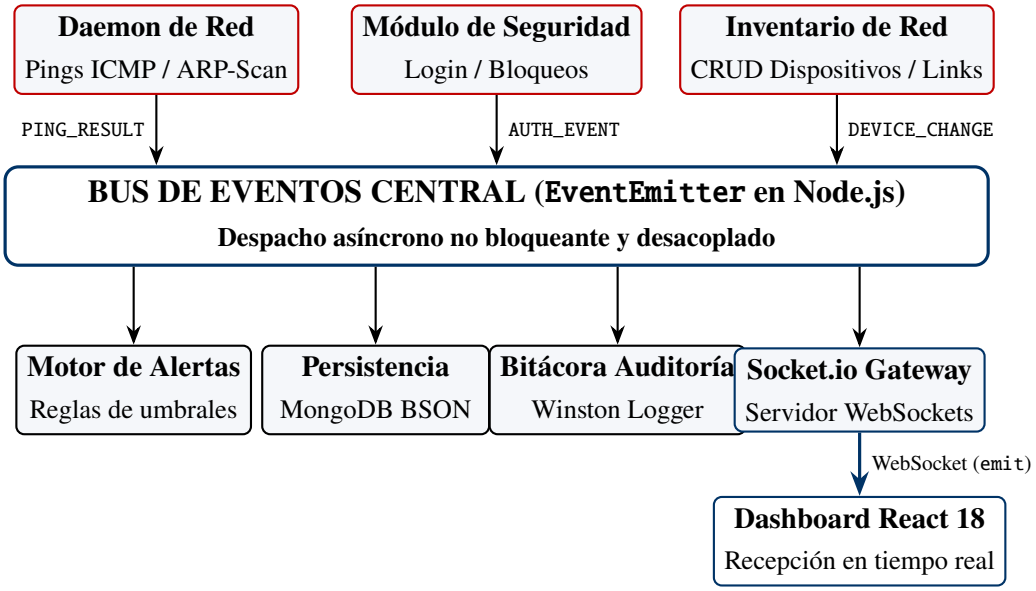


Figura 2.1: Flujo arquitectural de la Arquitectura Dirigida por Eventos (EDA)

Capítulo 3

Desglose Exhaustivo del Stack Tecnológico

El sistema implementa una arquitectura modular desacoplada basada en eventos, alineándose con las directrices de la norma ISO/IEC 25010 en cuanto a modularidad, mantenibilidad, fiabilidad y eficiencia de desempeño.

3.1 Capa de Presentación: Frontend Web Reactivo

La interfaz gráfica de usuario está orientada a la operación en salas de control y centros de soporte de redes (NOC). Debe responder fluidamente en pantallas de escritorio con resolución igual o superior a 1366×768 píxeles (RNF-10).

- **Librería Base de Interfaz: React 18.2+**. Proporciona renderizado reactivo concurrente mediante Virtual DOM. Permite actualizar selectivamente widgets de métricas y nodos de red sin recargas de página.
- **Herramienta de Construcción y Empaquetado: Vite 5.x**. Ofrece arranque ultraveloz en desarrollo basado en precompilación con *esbuild* y empaqueta el producto final en producción con *Rollup*, reduciendo drásticamente el peso de las transferencias.
- **Comunicación Bidireccional en Tiempo Real: Socket.io Client (v4.7+)**. Mantiene una conexión persistente sobre WebSockets con reconexión automática y retrocompatibilidad (*polling fallback*). Recibe en tiempo real los eventos de telemetría y alertas emitidos por el backend.
- **Cliente HTTP Transaccional y Caché: Axios y TanStack Query (React Query v5)**. Administra las consultas REST síncronas (autenticación, formularios, reportes históricos) con invalidación controlada de caché, evitando peticiones duplicadas y manejando cancelaciones de solicitudes.
- **Renderizado del Grafo de Topología: React Flow (@xyflow/react v12)**. Librería especializada en modelado de redes interactivas sobre Canvas/SVG. Permite representar switches, routers y terminales con íconos vectoriales, enlaces físicos continuos o discontinuos, controles de arrastre, zoom infinito y selección de nodos para inspección de métricas.
- **Sistema de Diseño y Estilos: Tailwind CSS v3.4**. Framework de clases utilitarias que garantiza adherencia a la paleta institucional, consistencia de espaciado y tipografía, y respuesta fluida en diferentes resoluciones.
- **Iconografía de Alta Definición: Lucide React**. Set moderno de íconos vectoriales que refuerza el estado de los nodos con señales gráficas explícitas (check, alerta, cruz), evitando depender únicamente del color para cumplir la norma WCAG 2.1 AA.

3.2 Capa de Servicios y Lógica de Negocio: Backend Node.js

El servidor de aplicaciones procesa las transacciones, gestiona la seguridad y ejecuta el bus de mensajería interna.

- **Entorno de Ejecución: Node.js v20 LTS (Iron)**. Plataforma basada en el motor V8 de Google que ejecuta un bucle de eventos no bloqueante (*Event Loop*), idónea para operaciones concurrentes de E/S de red y persistencia sin levantar hilos pesados del sistema operativo.
- **Framework Web y Middlewares: Express.js v4.21+**. Microframework minimalista que organiza la capa HTTP mediante middlewares de seguridad (*Helmet*), compresión (*compression*), deserialización segura (*express.json*) y auditoría de peticiones (*Morgan*).
- **Servidor de WebSockets: Socket.io v4.8+**. Servidor de comunicación en tiempo real acoplado al servidor HTTP de Node.js. Administra salas de transmisión (*rooms*) para segmentar el tráfico de eventos según la vista activa del operador.
- **Bus Central de Eventos: Node.js EventEmitter (Módulo Interno)**. Patrón de publicación y suscripción (*Pub/Sub*) en memoria que desacopla la recolección de métricas de su distribución y almacenamiento.
- **Seguridad Criptográfica: bcrypt (12 rondas de salting)**. Función hash unidireccional y adaptativa que protege las contraseñas contra ataques de fuerza bruta y tablas arcoíris. No se almacenan contraseñas en texto plano (RNF-05).
- **Gestión de Sesiones Seguras: jsonwebtoken (JWT Dual)**. Acceso mediante dos tokens diferenciados:
  - *Access Token:* JWT de corta vigencia (15 minutos), firmado con HMAC-SHA256, transportado en la cabecera HTTP `Authorization: Bearer`.
  - *Refresh Token:* Credencial de larga duración (8 horas), almacenada en base de datos y transferida exclusivamente mediante cookie con banderas `HttpOnly`, `Secure` y `SameSite=Lax`, mitigando de raíz ataques XSS y CSRF.
- **Validación Defensiva de Esquemas: Zod v3.24+**. Validación rigurosa de entradas en tiempo de ejecución. Impide la inyección de tipos arbitrarios y restringe umbrales y tiempos de sondeo a los límites seguros de la especificación (RF-10).

3.3 Capa de Monitoreo y Exploración de Red: Daemon Centinela

El proceso de monitoreo opera de forma totalmente independiente del servidor web y de las sesiones activas en el navegador (RNF-03).

- **Arquitectura del Proceso:** Servicio nativo en segundo plano orquestado mediante **Systemd** en el servidor Debian del laboratorio. Si el proceso sufriera una falla inesperada, Systemd lo reinicia automáticamente en menos de 5 segundos con política `Restart=always`.
- **Mecanismos de Descubrimiento de Red (Capa 2 y Capa 3):**

  - **arp-scan:** Herramienta nativa de Linux que envía tramas ARP a nivel de Capa de Enlace (Ethernet). Permite mapear instantáneamente todos los dispositivos activos en el segmento local del laboratorio, obteniendo su dirección IP y su dirección MAC física real de hardware, incluso si el dispositivo tiene activado un firewall personal que bloquea paquetes ICMP.
  - **Nmap (Network Mapper):** Empleado para barridos de subred configurados (`nmap -sn`) y para resolución pasiva de fabricantes mediante consultas a la base de datos de identificadores OUI del IEEE.
- **Mecanismos de Comprobación de Disponibilidad:**
  - **Pings ICMP de baja sobrecarga:** Ejecución de comprobaciones de eco con tiempo de espera configurable (predeterminado 3 segundos, timeout estricto de 1 paquete).
  - **Sondeo de puertos TCP (Layer 4 fallback):** Comprobación de sockets TCP a puertos estándar (ej. 80, 443, 22) para servidores que no responden a eco ICMP pero prestan servicios de red activos.
- **Control de Concurrencia y Consumo de Recursos (RNF-15):** Limitación estricta de concurrencia mediante un pool de trabajadores asíncronos (`p-limit`) configurado a un máximo de 15 comprobaciones simultáneas. Esto impide la saturación de sockets de red y el agotamiento de descriptores de archivos en el núcleo de Linux.

3.4 Capa de Persistencia: Base de Datos NoSQL

La capa de datos utiliza un motor flexible para acomodar la naturaleza heterogénea de los equipos de red (hosts con IP, switches no administrables sin IP, puntos de acceso con múltiples clientes).

- **Motor de Base de Datos: MongoDB 7.0 Community Edition**. Sistema de base de datos documental orientado a JSON/BSON. Permite almacenar atributos dinámicos por tipo de hardware sin requerir complejas migraciones de esquema relacional.
- **Mapeador Objeto-Documento (ODM): Mongoose 8.9+**. Garantiza tipado estricto en el código, validación de reglas de negocio a nivel de esquema, manejo de índices compuestos únicos y enganches (*hooks*) previos a la persistencia.
- **Estrategia de Depuración Automática (Índices TTL):**
  - Colección `sessions`: Los registros de sesión expiran y se eliminan automáticamente según el campo `expiresAt`.
  - Colección `metrics`: Las métricas de ping de alta resolución se consolidan por promedios horarios y expiran a los 30 días para evitar el desbordamiento de disco (RNF-13).
  - Colección `events`: Registro histórico de eventos e incidencias con retención de 90 días (equivalente al período académico).

3.5 Infraestructura del Host y Despliegue en Servidor Físico

El proyecto se aloja directamente en el servidor de laboratorio asignado, garantizando acceso nativo a la interfaz de red física:

- **Sistema Operativo Base: Debian GNU/Linux 12 (Bookworm)**. Distribución conocida por su máxima estabilidad, mínima superficie de ataque y soporte nativo de utilidades de red avanzadas.
- **Servidor Web Inverso: Nginx 1.22+**. Actúa como puerta de enlace frontal, gestionando la terminación segura de certificados SSL/TLS, compresión estática *gzip*, y canalizando peticiones HTTP ordinarias y actualización de conexiones WebSockets (`Upgrade: websocket`) hacia el proceso Node.js interno.
- **Gestión de Procesos en Producción: Systemd**. Se configuran unidades de servicio dedicadas (`noc-backend.service`) para arranque automático con el sistema operativo, aislamiento mediante usuarios sin privilegios (`nocuser`) y rotación automática de bitácoras de consola hacia `journald`.
- **Acceso Remoto para Demostraciones y Mantenimiento: Cloudflare Tunnel / SSH inverso**. Permite exponer de manera cifrada el panel de control durante sesiones de evaluación técnica y clases sin abrir puertos del sistema a Internet ni comprometer la seguridad institucional.

Capítulo 4

Matriz de Justificación Técnica y Alternativas Descartadas

Para cumplir con los estándares de rigor técnico en la toma de decisiones de ingeniería de software, se documenta la evaluación comparativa frente a tecnologías alternativas dividida por ámbitos de diseño.

Tabla 4.1: Matriz comparativa de arquitectura, tiempo real y desarrollo web

Capa / Componente	Tecnología Elegida	Alternativas Evaluadas	Criterio y Justificación de Decisión
Patrón Arquitectural	**Monolito Modular Dirigido por Eventos (EDA)**	Microservicios distribuidos, Monolito síncrono N-capas	Microservicios introduce sobrecarga de latencia de red entre servicios y complejidad innecesaria para un equipo de 4 personas. El monolito síncrono no puede responder en tiempo real a eventos de red. EDA modular ofrece el equilibrio óptimo entre desacoplamiento y simplicidad operativa.
Comunicación en Tiempo Real	**WebSockets (Socket.io)**	HTTP Polling periódico, Server-Sent Events (SSE)	HTTP Polling satura la red con peticiones vacías cada pocos segundos y genera latencia inaceptable. SSE solo permite flujo unidireccional servidor-cliente. Socket.io ofrece transporte bidireccional, reconexión inteligente, salas de aislamiento y latencia inferior a 50 ms.
Frontend Web	**React 18 + Vite**	Angular, Vue.js, Plantillas Server-Side (EJS/Django)	React cuenta con el ecosistema de visualización de grafos de red más maduro (@xyflow/react). Vite ofrece compilaciones ultrarrápidas y menor consumo de memoria en desarrollo que Webpack.
Backend & API	**Node.js + Express**	Python (Django/FastAPI), Java (Spring Boot)	El modelo de I/O asíncrono no bloqueante de Node.js es superior al manejar cientos de comprobaciones concurrentes de red sin levantar hilos pesados del sistema operativo.

Tabla 4.2: Matriz comparativa de persistencia, monitoreo de red e infraestructura

Capa / Componente	Tecnología Elegida	Alternativas Evaluadas	Criterio y Justificación de Decisión
Base de Datos	**MongoDB 7.0**	PostgreSQL, MySQL, SQLite	La estructura de los dispositivos de red es polimórfica (un router tiene tablas de rutas; un switch puertos y VLANs; una PC no). Los documentos BSON modelan esta jerarquía sin tablas relacionales artificiales. Dispone de índices TTL nativos.
Daemon de Red	**Node.js Daemon + arp-scan/ping**	Python Scapy, C/C++ raw sockets	‘arp-scan‘ y ‘ping‘ son binarios del sistema altamente optimizados por el kernel de Linux. Integrarlos con Node.js proporciona equilibrio entre velocidad de ejecución en bajo nivel y facilidad de lógica de negocio.
Despliegue	**Systemd + Nginx en Debian**	Docker Compose en VM, Kubernetes	En la máquina física del laboratorio asignada para el proyecto, capas pesadas de virtualización consumen memoria RAM innecesaria. La ejecución como servicio nativo Systemd garantiza el 100 % del rendimiento del hardware.

Capítulo 5

Articulación con las Asignaturas del Semestre (PAO 5)

El stack tecnológico seleccionado y la arquitectura dirigida por eventos se articulan de manera interdisciplinaria para integrar las competencias prácticas exigidas por las asignaturas de la carrera:

Tabla 5.1: Mapeo de tecnologías con asignaturas curriculares

Asignatura y Docente	Componentes del Stack Aplicados	Aporte Específico al Proyecto
Infraestructura TI (Ing. Tatiana Landivar)	Servidor Debian 12, Daemon Systemd, Nginx, Gobernanza de Servidores	Despliegue de servicios en Linux, gestión de dependencias, automatización de tareas en segundo plano y políticas de seguridad perimetral del host.
Tecnología Web (Ing. Pedro Aguilar)	React 18, Vite, Node.js, Express, WebSockets (Socket.io), Arquitectura EDA, JWT dual tokens	Arquitectura reactiva dirigida por eventos, canales WebSocket de telemetría bidireccional, protección de sesiones web y diseño modular de servicios.
Conmutación y Enrutamiento (Ing. Juan Pazmiño)	arp-scan, ICMP, sondeos TCP, Capa 2/3, Nodos Parent-Child	Comprensión de tramas Ethernet broadcast, direccionamiento IP, pasarelas por defecto, tablas de conmutación y correlación de fallos de red.
Big Data (Ing. Geovanny Silva)	MongoDB, Índices TTL, Agregación Downsampling, Métricas históricas	Procesamiento de grandes volúmenes de telemetría de red, agregación diaria de disponibilidad y retención eficiente sin desbordamiento de almacenamiento.
Diseño y Tecnología Multimedia (Ing. Ramiro Bonilla)	React Flow (@xyflow), Tailwind CSS, Lucide Icons, WCAG 2.1 AA	Visualización de grafos interactivos de topología, ergonomía visual, centro de notificaciones con accesibilidad cromática y acústica.

Capítulo 6

Conclusiones

La selección del stack tecnológico y la adopción de una Arquitectura Dirigida por Eventos (EDA) responden a criterios de ingeniería rigurosos: notificación en tiempo real con latencia inferior a 50 milisegundos, mínimo consumo de recursos en hardware local, desacoplamiento absoluto entre la recolección de métricas y su visualización, y cumplimiento integral de las normas de calidad ISO/IEC 25010 y de seguridad ISO/IEC 27001. Este documento constituye la base técnica vinculante sobre la cual se desarrollará la solución durante el período académico.